

Context

A hands-on course in building agents by building their harness

Lesson 4

The model knows nothing about this moment

The model knows nothing about this moment except what's in the request. It doesn't know where it's running, what day it is, who it is, what it did a minute ago, or what you told it last week, unless the harness puts it there.

Every call starts from nothing

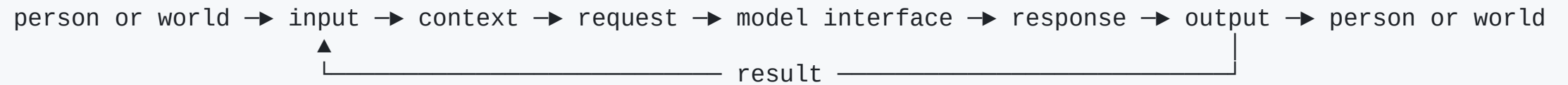
Every call starts from nothing.

Context

Context is how inputs are presented to the model. Input gathers what goes in; context decides what the request actually holds and how it's laid out.

Where it sits

Here's where it sits on the path from Lesson 3:



Context does two things

Context does two things before every call. It **assembles**: it chooses what goes in the request and how it's presented.

It fits

And it **fits**: the model can only read so many tokens at once, its context window, so when what you'd send is bigger than that, something has to give.

The components

Almost everything people build into harnesses is a context component, because almost everything is a way of deciding what the model sees:

- **Instructions:** who the model is, what it's for, how to behave. Usually a system prompt.
- **Working memory:** what's happened in this session.
- **Episodic memory:** a record of what happened in past sessions.
- **Semantic memory:** facts that outlast a session: who you are, what you prefer, what went wrong last time.

The components, continued

- **Procedural memory:** how to do things. Recipes, playbooks, skills read when they're relevant.
- **Retrieval:** search a store and put what you found in the request.
- **Compaction:** when working memory won't fit, replace some of it with a summary.
- **Self-knowledge:** tell the model what it is and how it works.

You don't need all of them

You don't need all of them. You need the ones your agent needs, built in whatever way fits.

The worked example

Here's Lesson 3's agent with quark's context added.

It's `quark.py`, 234 lines: 82 of code, and a system prompt of 152 lines.

The prompt is shortened to `...` here and shown in full after the code:

The code: model interface and output

```
import subprocess, sys, os, re, glob, json, datetime
from anthropic import Anthropic, BadRequestError

# — model interface —————
client = Anthropic()
MODEL = "claude-sonnet-5-5"
def call(**request): return client.messages.create(model=MODEL, **request)

# — output: the one tool —————
tools = [{"name": "bash", "description": "Run shell command – the whole system is in reach", "input_schema": {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]
```

The code: episodic memory

```
# — context —————  
EPISODE = f".quark/episodes/{datetime.datetime.now():%Y-%m-%dT%H-%M-%S}.jsonl"  
  
def remember(message):                                # episodic memory: the harness writes every message, as it was  
    os.makedirs(os.path.dirname(EPISODE), exist_ok=True)  
    with open(EPISODE, "a") as f: f.write(json.dumps(message, default=lambda b: b.model_dump(exclude_none=True)) + "\n")  
  
def add(working_memory, message):                      # one message, two places: in context and on disk  
    working_memory.append(message); remember(message)
```

The code: procedural memory

```
def skills():                                # procedural memory: an index built from each skill's front matter
    index = []
    for path in sorted(glob.glob(".quark/skills/*.md")):
        text = open(path).read()
        name, about = (re.search(rf"^{key}:\s*(.+)$", text, re.M) for key in ("name", "description"))
        index.append(f"- {name[1] if name else os.path.basename(path)}: {about[1] if about else '(no description)'} ({path})")
    return "\n".join(index) or "- (none yet)"
```

The code: self-knowledge and instructions

```
def mechanics():  
    # self-knowledge: this file, with the system prompt redacted  
    return re.sub(r"^def system\\(\\):.*?(?=^def )", "def system(): ... # redacted: it is the prompt you are reading\\n\\n", open(__file__).read(), flags=re.S | re.M)  
  
def system():  
    # instructions: the prompt is shown in full below  
    return [{"type": "text", "cache_control": {"type": "ephemeral"}, "text": f"\"\"\"...\"\"\"}]
```

The code: compaction

```
def compact(working_memory, drop):  
    # lazy: runs only after the API says the prompt is too long  
    turns = [i for i, m in enumerate(working_memory) if m["role"] == "user" and isinstance(m["content"], str)]  
    if drop > len(turns): sys.exit("[working memory can't be summarized small enough]")  
    keep = working_memory[turns[drop]:] if drop < len(turns) else [working_memory[turns[-1]]]  
    summary = call(max_tokens=2048, system=system(), messages=keep + [{"role": "user", "content": "Your working memory is full. Summarize into a gist that preserves what matters for continuing."}])  
    gist = next((b.text for b in summary.content if b.type == "text"), "")  
    return [{"role": "user", "content": f"[your earlier working memory, summarized; every original message is in {EPISODE}] {gist}"]}]
```

The code: input

```
# — input —————  
def read(prompt):                                # input: from a person  
    while True:  
        print(prompt, end="", flush=True)  
        line = sys.stdin.readline()  
        if not line: return "/q"                # end of input (Ctrl-D): nothing more is coming  
        if line.strip(): return line.rstrip("\n") # Enter on an empty line: a fresh prompt, as in a terminal  
        prompt = "> "  
input = " ".join(sys.argv[1:]) or read("> ")  
if input == "/q": sys.exit()  
chat = len(sys.argv) < 2
```

The code: the loop, up to the call

```
# — control flow —————  
working_memory, drop = [], 0  
add(working_memory, {"role": "user", "content": input})  
  
while True:  
    try:  
        if drop:  
            working_memory, drop = compact(working_memory, drop), 0  
        output = call(max_tokens=16384, system=system(), tools=tools, messages=working_memory).content  
    except BadRequestError as e:  
        if "prompt is too long" not in str(e): raise  
        drop += 1  
        continue
```

The code: the loop, after the call

```
add(working_memory, {"role": "assistant", "content": output}) # on disk before any tool runs

input = []
for block in output: # output: show text, run tool requests
    if block.type == "text":
        print(block.text)
    if block.type == "tool_use":
        print(f"$ {block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
        print(done.stdout)
        input.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit {done.returncode})"}) # input: from the world
```

The code: the loop, the end

```
if input:
    add(working_memory, {"role": "user", "content": input})
    continue
if not chat or (input := read("\n> ")) == "/q":
    break
add(working_memory, {"role": "user", "content": input})
```

Lesson 3's, with two changes

Everything outside `# — context —` is Lesson 3's, with two changes in the loop. The `messages` list is renamed `working_memory`, and every `messages.append(...)` becomes `add(working_memory, ...)`. The call is now wrapped in `try`, so quark can catch "prompt is too long".

Eight components

The new section is context, and it has eight components.

Working memory

Lesson 3's `messages` list, renamed `working_memory`, because that's what it is: everything that has happened in this session, sent in full on every call. It's how the model knows what it did two passes ago.

Episodic memory: `EPISODE`, `remember()` and `add()`

Every message is written twice, by the harness: into working memory, and as one line of `.quark/episodes/<start time>.jsonl`, one file per session.

An episode is what working memory would have been

The line is the message exactly as it was in working memory, so an episode is what working memory would have been if nothing had ever been dropped.

Written before any tool runs

The model's output is written before any tool runs, so a session that dies mid-command still shows what was asked. The model never writes episodes; it only reads them.

Instructions: `system()`

The system prompt, sent as `system=system()` on every call and rebuilt each time, so the directory, the date and the skill index are current.

The date, not the time

It holds the date but not the time because `cache_control` asks the API to cache the prompt, and a cache only hits when the prompt is identical.

The prompt is models of what quark needs to know

It's written as models of what quark needs to know: a **Self Model**, **Memory**, a **World Model**, an **Other Selves Model**, **Body Operations** and **Mechanics**.

Self-knowledge: `mechanics()`

quark reads its own file and puts it at the end of the system prompt, so the model can see the harness it runs in. The `system()` function is swapped for a one-line placeholder, since the model is already reading the prompt.

Semantic memory:

. quark/memory/memory .md

Facts that last, one per line, filed under a subject: `- <subject>: <fact> .` There's no memory code: the prompt gives the exact commands to create the file, write a fact and replace one, and the moves for reading it.

Semantic facts carry no time

Semantic facts carry no time. Each line is what's true now; when it was learned lives in episodic memory.

Distill

The prompt asks quark to **distill**: keep the general truth behind what happened, not a record of it, one truth per line.

Procedural memory: `.quark/skills/` and `skills()`

How to do things, one Markdown file per skill, with a `name` and a `description` at the top. quark writes them with the exact command in the prompt, and the prompt asks it to **generalize**: a skill is the method behind a task that worked, with the particulars replaced by placeholders, named for the whole class of tasks it serves.

`skills()` is harness code

`skills()` is harness code: it reads every skill's header and puts the index in the prompt, so the model sees what it knows how to do and reads a whole skill only when a task matches.

Recall across memories

The prompt orders the stores into a ladder, from the most distilled to the most complete: semantic, then procedural, then episodic. quark stops as soon as it has what it needs.

Moves across stores

It also gives moves across stores: search all three at once, follow a fact to its skill and a skill to the sessions that used it, and find where a fact came from by searching the episodes, the only store with time in it.

Compaction: `compact()`

When working memory grows past what the model can read, the API refuses the request with "prompt is too long". quark catches that and adds one to `drop`.

How `compact()` works

On the next pass, `compact()` drops the oldest `drop` turns, asks the model to summarize what's left, and replaces working memory with the summary. If even the summary request is too long, `drop` goes up and it tries again with less.

Lazy, and it loses nothing

This is *lazy*: quark never spends a call on a summary it didn't need. And it loses nothing, because the summary says where the originals are, and every one of them is still in the episode file.

You're unlikely to see it happen

You're unlikely to see it happen: this model can read about a million tokens, so a session has to run very long to fill it.

The four memories

Memory	Who writes it	Where	How the model sees it
Working	the harness, <code>add()</code>	<code>working_memory</code>	every call, as the messages
Episodic	the harness, <code>add()</code>	<code>.quark/episodes/<start>.jsonl</code>	by searching, when the past matters
Semantic	the model, with bash	<code>.quark/memory/memory.md</code>	by reading, when facts matter
Procedural	the model, with bash; indexed by <code>skills()</code>	<code>.quark/skills/<name>.md</code>	the index every call; a skill when it applies

The system prompt

Its sections:

```
# Self Model
# Memory
## Semantic memory – facts that last
## Procedural memory – how to do things
## Episodic memory – what happened
## Across memories
# World Model
# Other Selves Model
# Body Operations
# Mechanics
```

Each memory store gets the same parts

Each memory store gets the same parts: the store, how to initialize it, the format to preserve, the exact command to write it with placeholders to fill in, what's worth writing, the read moves, and its rules.

The read moves are "moves, not a menu": quark composes whatever text tools answer the question.

From the whole prompt: write a fact

```
Write (the quoted heredoc keeps the fact literal):  
cat >> .quark/memory/memory.md << 'EOF'  
- <subject>: <fact>  
EOF
```

Left out of the course version

quark's own version also lets you interrupt it with ESC, and retries the summary if the network fails. Those are hardening, so they're left out here.

Run it

From the root of the repo, ask it what it is:

```
uv run lessons/04-context/quark.py "what are you, and how do you work? three sentences"
```

Run it: one run

Here's one run:

I'm quark, an agent that runs on a language model and acts only through bash. It's my one tool for reading, writing, running programs and talking to you. I work in a loop of observe, think, act, and repeat. My working memory is this session's context window. Three stores on disk persist beyond it: semantic facts in `~/.quark/memory/memory.md`, skills in `~/.quark/skills/`, and a log of every past session in `~/.quark/episodes/`.

It knows what it is

It knows its name, its body, its loop and its memories because they're in its context.

Tell it something

Now tell it something:

```
uv run lessons/04-context/quark.py "remember that I prefer short answers"
```

It writes a fact

```
$ mkdir -p .quark/memory && [ ! -f .quark/memory/memory.md ] && echo "# Quark Memory" > .quark/memory/memory.md; grep -i "^- user:" .quark/memory/memory.md; cat >> .quark/memory/memory.md << 'EOF'
- user: prefers short answers
EOF
```

Noted. I'll keep answers short.

A new run, empty working memory

That run has ended. A new one starts with empty working memory:

```
uv run lessons/04-context/quark.py "what do you know about me?"
```

What it reads

```
$ cat .quark/memory/memory.md 2>/dev/null; ls .quark/skills 2>/dev/null; ls .quark/episodes 2>/dev/null | head
# Quark Memory
- user: prefers short answers
2026-10-06T16-30-30.jsonl
2026-10-06T16-30-34.jsonl
2026-10-06T16-30-39.jsonl

$ grep -m1 -H "" .quark/episodes/*.jsonl | grep -v 16-30-39 | cut -c1-250
.quark/episodes/2026-10-06T16-30-30.jsonl:{"role": "user", "content": "what are you, and how do you work? three sentences"}
.quark/episodes/2026-10-06T16-30-34.jsonl:{"role": "user", "content": "remember that I prefer short answers"}
```

What it says

Not much:

- You prefer short answers. This is the only fact in my memory.
- Earlier sessions show you asked me to describe what I am and how I work in three sentences. You also asked me to remember your short-answer preference.

I don't know your name, your work, or anything else about you.

What carried over

Semantic memory carried the fact, and episodic memory carried what happened.
Working memory carried nothing between the runs.

Give it something that will come up again

Now give it something that will come up again:

```
uv run lessons/04-context/quark.py "I'll often ask you how many lines of Python are in a repo, not counting .venv. Do it for this one now."
```

It counts

```
$ cd /home/user/building-agents && find . -name '*.py' -not -path '**/.venv/**' -not -path './.git/**' -print0 | xargs -0 cat | wc -l; find . -name '*.py' -not -path '**/.venv/**' -not -path './.git/**' | wc -l
1949
24
```

It writes a skill and a fact

```
$ mkdir -p .quark/skills .quark/memory && cat > .quark/skills/count-python-lines.md << 'EOF'
---
name: count-python-lines
description: count how many lines of Python are in a repo, excluding .venv (and .git)
---
1. cd to the repo root (default: the current working directory).
2. Run: find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' -print0 | xargs -0 cat | wc -l
3. Optionally count files: same find piped to wc -l.
4. Report total lines and number of files.
EOF
[ -f .quark/memory/memory.md ] || echo "# Quark Memory" > .quark/memory/memory.md
cat >> .quark/memory/memory.md << 'EOF'
- user: often asks how many lines of Python are in a repo, excluding .venv (skill: count-python-lines)
EOF
```

It answers

This repo has **1,949** lines of Python across 24 `.py` files. The count excludes `.venv` and `.git`.

I also saved the method as a skill, `count-python-lines`, so I can repeat it quickly next time.

It wrote two memories

It answered, and it wrote two memories: a skill, named for the whole class of task and not for this repo, and a fact about you that points to the skill.

In a new session

In a new session:

```
uv run lessons/04-context/quark.py "how many lines of Python are in this repo?"
```

It reads the skill

```
$ cat .quark/skills/count-python-lines.md 2>/dev/null
---
name: count-python-lines
description: count how many lines of Python are in a repo, excluding .venv (and .git)
---
1. cd to the repo root (default: the current working directory).
2. Run: find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' -print0 | xargs -0 cat | wc -l
3. Optionally count files: same find piped to wc -l.
4. Report total lines and number of files.
```

It follows the skill

```
$ find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' -print0 | xargs -0 cat | wc -l; find . -name '*.py' -not -path '*/.venv/*' -not -path './.git/*' | wc -l
1949
24
```

The repo has **1,949 lines of Python** across **24 files**. I excluded `.venv`` and `.git`` from the count.

The skill was in the index

The skill was in the index in its prompt, so the first thing it did was read the skill and follow it.

A question only episodic memory can answer

Last, a question only episodic memory can answer:

```
uv run lessons/04-context/quark.py "what have I asked you in earlier sessions?"
```

It lists the episodes

```
$ ls .quark/episodes/ 2>/dev/null | head -50; grep -m1 -H "" .quark/episodes/*.jsonl 2>/dev/null | grep -v 2026-10-06T16-31-00 | cut -c1-250
2026-10-06T16-30-30.jsonl
2026-10-06T16-30-34.jsonl
2026-10-06T16-30-39.jsonl
2026-10-06T16-30-45.jsonl
2026-10-06T16-30-53.jsonl
2026-10-06T16-31-00.jsonl
```

It reads each opening input

```
.quark/episodes/2026-10-06T16-30-30.jsonl:{"role": "user", "content": "what are you, and how do you work? three sentences"}  
.quark/episodes/2026-10-06T16-30-34.jsonl:{"role": "user", "content": "remember that I prefer short answers"}  
.quark/episodes/2026-10-06T16-30-39.jsonl:{"role": "user", "content": "what do you know about me?"}  
.quark/episodes/2026-10-06T16-30-45.jsonl:{"role": "user", "content": "I'll often ask you how many lines of Python are in a repo, not counting .venv. Do it for this one now."}  
.quark/episodes/2026-10-06T16-30-53.jsonl:{"role": "user", "content": "how many lines of Python are in this repo?"}
```

It recalls what you asked

In earlier sessions you asked me:

1. "what are you, and how do you work? three sentences"
2. "remember that I prefer short answers"
3. "what do you know about me?"
4. "I'll often ask you how many lines of Python are in a repo, not counting .venv. Do it for this one now."
5. "how many lines of Python are in this repo?"

The move the prompt gives it

It used the move the prompt gives it: the first line of every episode is the input that opened that session, so one `grep` lists them all, leaving out the session it's in.

What else context can be

quark keeps working memory, a system prompt, its own source, three memory stores and lazy compaction.

The choices you make

The components listed at the top of this lesson are what context is made of; these are the choices you make when you build them.

- **Who decides what's remembered.** The model, writing it down when it thinks it matters, or harness code that saves and loads on its own.
- **What it retrieves.** Nothing, or documents, code or past messages found by search and put in the request.
- **When it fits.** After the API refuses, or before, by counting tokens against a limit.
- **How it fits.** Summarize old turns, drop them, or cut long tool results down before they're kept.
- **How it's laid out.** One system prompt, or instructions loaded only when a task needs them.

A product built on context

It can be a product on its own. Memory layers like [Mem0](#) are this primitive: they store what an agent should remember and hand back what's relevant for each request.

A fuller example: `context.py`

`context.py` makes different choices from quark on several of these, so you can see the alternatives side by side:

context.py: setup

```
import subprocess, sys, os, json, glob, datetime
from anthropic import Anthropic

client = Anthropic()
MODEL = "claude-sonnet-5-5"
tools = [{"name": "bash", "description": "Run shell command", "input_schema": {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]}
LIMIT, KEEP = 50_000, 4_000
```

context.py: the prompt

```
def system():
    skills = "\n".join(f"- {p}: {open(p).readline().strip()}" for p in sorted(glob.glob(".quark/skills/*.md"))) or "(none yet)"
    return f"""You are quark, an agent whose body is bash. You're in {os.getcwd()}, and today is {datetime.date.today()}.
Past sessions are logged one message per line in .quark/episodes.jsonl. Search it when the past matters.
Skills: before a task one of these covers, read it with cat and follow it.
{skills}"""
```

context.py: the log

```
def remember(message):  
    os.makedirs(".quark", exist_ok=True)  
    with open(".quark/episodes.jsonl", "a") as f: f.write(json.dumps(message, default=lambda b: b.model_dump()) + "\n")
```

context.py: trim()

```
def trim(text):  
    return text if len(text) <= KEEP else f"{text[:KEEP // 2]}\n[... {len(text) - KEEP} characters cut ...]\n{text[-KEEP // 2:]}"
```

context.py: fit()

```
def fit(working_memory):
    if client.messages.count_tokens(model=MODEL, system=system(), tools=tools, messages=working_memory).input_tokens < LIMIT: return working_memory
    turns = [i for i, m in enumerate(working_memory) if m["role"] == "user" and isinstance(m["content"], str)]
    if len(turns) < 2: return working_memory
    old, recent = working_memory[:turns[-1]], working_memory[turns[-1]:]
    summary = client.messages.create(model=MODEL, max_tokens=2048, messages=old + [{"role": "user", "content": "Summarize this session so far into a gist that preserves what matters for continuing."}])
    gist = next((b.text for b in summary.content if b.type == "text"), "")
    print(f"working memory over {LIMIT} tokens: summarized {turns[-1]} messages")
    return [{"role": "user", "content": f"[earlier in this session, summarized] {gist}"}] + recent
```

context.py: add()

```
def add(working_memory, message):  
    working_memory.append(message); remember(message)
```

context.py: input

```
task = " ".join(sys.argv[1:]) or input("> ")
chat = len(sys.argv) < 2
working_memory = []
add(working_memory, {"role": "user", "content": task})
```

context.py: the loop, up to the tools

```
while True:
    working_memory = fit(working_memory)
    reply = client.messages.create(model=MODEL, max_tokens=16384, system=system(), tools=tools, messages=working_memory)
    results = []
    for block in reply.content:
        if block.type == "text": print(block.text)
        if block.type == "tool_use":
            print(f"$ {block.input['cmd']}")
            done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
```

context.py: the loop, the end

```
print(done.stdout)
results.append({"type": "tool_result", "tool_use_id": block.id, "content": trim(done.stdout) or f"(exit {done.returncode})"})
add(working_memory, {"role": "assistant", "content": reply.content})
if results:
    add(working_memory, {"role": "user", "content": results}); continue
if not chat or (task := input("\n> ")) == "/q": break
add(working_memory, {"role": "user", "content": task})
```

When it fits

When it fits. quark waits for the API to refuse. `fit()` counts tokens before every call, and past `LIMIT`, a budget you choose, it summarizes everything before your latest message and keeps the rest as it is. It costs a count per call, and it keeps every request smaller and cheaper than the model's limit.

How it fits

How it fits. quark keeps every tool result whole. `trim()` keeps the start and end of a result longer than `KEEP` characters and cuts the middle, before the result goes into working memory.

Episodic memory

Episodic memory. quark writes one file per session. `context.py` writes every message of every session to one shared log, `.quark/episodes.jsonl`: simpler, and harder to search once it grows.

Procedural memory

Procedural memory. quark indexes skills by a `name` and `description` header. `context.py` lists each file by its first line: less structure, and nothing to keep in a format.

Try procedural memory

To try procedural memory, give it a skill:

```
mkdir -p .quark/skills
printf 'How to count lines of Python in this repo\n\nUse: find . -name "*.py" -not -path "./.venv/*" | xargs wc -l\nReport the total, then the largest file.\n' > .quark/skills/count-python.md
uv run lessons/04-context/context.py "how many lines of Python are in this repo?"
```

Here's one run

Here's one run:

```
$ cat .quark/skills/count-python.md  
How to count lines of Python in this repo  
  
Use: find . -name "*.py" -not -path "./.venv/*" | xargs wc -l  
Report the total, then the largest file.
```

It lists the files, first half

```
$ find . -name "*.py" -not -path "./.venv/*" | xargs wc -l | sort -n
  9 ./lessons/01-model-interface/quark.py
 16 ./lessons/01-model-interface/model_interface.py
 20 ./lessons/02-input-and-output/output.py
 21 ./lessons/03-control-flow/workflow.py
 24 ./lessons/02-input-and-output/input.py
 31 ./lessons/03-control-flow/control_flow.py
 33 ./lessons/02-input-and-output/quark.py
 46 ./lessons/03-control-flow/quark.py
 56 ./lessons/04-context/context.py
 59 ./production/05-observability/quark.py
 64 ./lessons/02-input-and-output/input_output.py
 66 ./paper/latex/build.py
```

It lists the files, second half

```
83 ./production/06-guardrails/quark.py
84 ./production/05-observability/observability.py
87 ./production/07-sandboxing/sandboxing.py
94 ./production/07-sandboxing/quark.py
98 ./production/06-guardrails/guardrails.py
100 ./production/09-performance/performance.py
112 ./production/08-resilience/resilience.py
122 ./production/10-evaluation/evaluation.py
135 ./production/08-resilience/quark.py
159 ./production/09-performance/quark.py
196 ./production/10-evaluation/quark.py
234 ./lessons/04-context/quark.py
1949 total
```

It answers by the skill

There are **1,949 lines** of Python in this repo. I excluded ``.venv/``, as the skill says to.

The largest file is ``lessons/04-context/quark.py``, at 234 lines.

It read the skill, then followed it

It read the skill, then followed it.

A past session, found

That session is now in the episode log, so a new one can find it:

```
uv run lessons/04-context/context.py "in a past session, how many lines of Python did you count? one line"
```

It searches its log

```
$ cd /home/user/building-agents; grep -i -n "lines" .quark/episodes.jsonl | head -50; cat .quark/skills/count-python.md
1:{ "role": "user", "content": "how many lines of Python are in this repo?" }
3:{ "role": "user", "content": [ { "type": "tool_result", "tool_use_id": "toolu_01G3mxni8y7p1gLWEpAR2HMZ", "content": "How to count lines of Python in this repo\n\nUse: find . -name \"*.py\" -not -path \"./.venv/*\" | xargs wc -l\n\nReport the total, then the largest file.\n\n" } ] }
6:{ "role": "assistant", "content": [ { "citations": null, "text": "There are **1,949 lines** of Python in this repo. I excluded \`.venv/`, as the skill says to.\n\nThe largest file is `lessons/04-context/quark.py`, at 234 lines.", "type": "text" } ] }
7:{ "role": "user", "content": "in a past session, how many lines of Python did you count? one line" }
```

It answers from the log

How to count lines of Python in this repo

```
Use: find . -name "*.py" -not -path "./.venv/*" | xargs wc -l
```

Report the total, then the largest file.

In a past session I counted 1,949 lines of Python, excluding ``.venv/``. The largest file was ``lessons/04-context/quark.py`` at 234 lines.

`fit()` at work

And with `LIMIT` set to 560 tokens, here's a chat where I typed four inputs (`my favorite color is green` , `what is 2+2?` , `what is 3+3?` and `what is my favorite color?` ; they aren't echoed). `fit()` summarizes before each of the last three, and the fact from the first survives in the summary:

```
> Noted: your favorite color is green. 📄🌿
```

```
Do you want me to save that somewhere, like a notes file in the repo, or was it just for me to know for this conversation?
```

```
> [working memory over 560 tokens: summarized 2 messages]
```

```
2 + 2 = 4.
```

`fit()` keeps the first fact

```
> [working memory over 560 tokens: summarized 3 messages]  
3 + 3 = 6.
```

The open question from earlier is still unanswered. Do you want your favorite color (green) saved somewhere, or should I just keep it in mind for this conversation?

```
> [working memory over 560 tokens: summarized 3 messages]  
Your favorite color is green, going by what you told me earlier in this session.
```

I also need to correct something from earlier. The summary says I have no files or tools, but I do have a bash shell in `~/home/user/building-agents``. I could save your preference there if you want. Should I save it, or keep it in this conversation only?

```
>
```

The rule

The rule: before every call, context assembles what the request holds and fits it in the space the model has. Instructions, self-knowledge, memory, retrieval and compaction are all ways of doing that.

Who writes each memory

Who writes each memory is a choice: the harness writes what happened, and the model writes what it distills from it.

What context never does

Notice what context never does. Getting the request to the model and the response back is the model interface. When to call, and whether a result goes back around, is control flow.

Input and output

Gathering what goes in is input, and handling the response is output.

Context only decides

Context only decides what the request holds and how it fits.

You've built a harness

Control flow, input, context, model interface, output. Five primitives in 234 lines, 82 of them code and the rest the prompt, and an agent that works, remembers facts, learns skills, recalls what happened, and knows what it is.

One set of choices

quark is one set of choices. Now you know what the choices are.

Go the other way

So go the other way. Pick a harness you haven't read. [nanoagent](#) is a good first one: another small agent, written in TypeScript. Or pick a big one.

Sort what you find

Read it and sort what you find under the five primitives:

control flow	what kind of loop? who decides when to stop?
└─ input	where do inputs come from: people, the world, both?
└─ context	what does the request hold? which memories? how does it fit?
└─ model interface	where does the request go, and how does the response come back?
└─ output	where do outputs go? which tools, and how are they run?

Some things won't fit at first

Some things won't fit at first. Ask what each one does. Is it deciding what the model sees? Then it's context, whatever it's called. Is it acting on what the model said? Output. Keep asking until it fits.

If it fits none of the five

If you find something that genuinely fits none of the five, I'd like to hear about it.

That's the primitives

That's the primitives. When you're ready to run your harness unattended, the production layers start with Lesson 5: Observability.

Next: Observability

Lesson 5